

# Brute Force Algorithms for the Dutch Merchant Problem

## Abstract

This document presents the design, implementation, and experimental analysis of three brute force algorithms for solving the Dutch Merchant Problem, corresponding to **Fase 3** of the project requirements. The algorithms include: pure brute force enumeration, brute force with pruning techniques, and a hybrid approach combining brute force route selection with dynamic programming for transaction optimization. These algorithms serve as exact solution methods for small instances, establishing optimal baselines for comparison with heuristic and approximation algorithms in **Fase 4**. The optimal solutions found here will be used as benchmarks to evaluate the quality and performance of efficient (non-exact) algorithms in subsequent phases of the project.

## 1 Introduction

This document corresponds to **Fase 3: Diseño de Soluciones Algorítmicas** of the project requirements. According to the project specifications, this phase requires designing and implementing a brute force algorithm that finds the optimal solution, which will serve as a baseline for comparing results in small instances.

The Dutch Merchant Problem, as formalized in the previous phase (Fase 2), is an NP-Complete optimization problem [4] that combines routing decisions (which ports to visit and in what order) with transaction optimization (what to buy and sell at each port). The problem is related to the Prize-Collecting Traveling Salesman Problem [9], which is known to be NP-hard [6]. Given the computational intractability of the problem for large instances, this phase focuses on developing exact algorithms that can solve small instances optimally.

The purpose of implementing brute force algorithms, as required by Fase 3, is twofold:

1. To establish optimal solutions for small instances, which will serve as benchmarks for evaluating the quality of heuristic and approximation algorithms in Fase 4. These optimal solutions are essential for measuring the approximation ratio and solution quality of efficient (non-exact) algorithms.
2. To empirically demonstrate the exponential growth in computational complexity, providing concrete evidence of the problem's intractability and justifying the need for heuristic approaches in Fase 4.

We present three progressively more efficient approaches:

- **Pure Brute Force:** Exhaustive enumeration of all routes and decision sequences, serving as the baseline optimal algorithm.
- **Brute Force with Pruning:** Same enumeration strategy enhanced with backtracking and feasibility pruning to eliminate infeasible branches early.
- **Hybrid (Brute Force + Dynamic Programming):** Separates route selection (brute force) from transaction optimization (dynamic programming), achieving significant performance improvements while maintaining optimality guarantees.

All three algorithms guarantee finding the optimal solution when they complete execution, but differ dramatically in their practical performance and the maximum instance size they can handle within reasonable time limits.

## 1.1 Single Commodity Constraint and Complexity Analysis

This implementation restricts the problem to a single commodity type per port ( $|M| = 1$ ), which significantly simplifies the decision space while preserving the essential combinatorial complexity of the problem.

### 1.1.1 Impact on Decision Space

With a single commodity, the decision space at each port is simplified to three discrete choices: {buy 1 unit, sell 1 unit, do nothing}. This reduces the decision space from potentially infinite (continuous quantities) to a manageable discrete enumeration. However, the combinatorial explosion still occurs due to:

1. **Route selection:**  $O(n!)$  permutations of port subsets
2. **Decision sequences:**  $O(3^k)$  for  $k$  intermediate ports
3. **Combined complexity:**  $O(n! \times 3^n)$  for pure brute force

### 1.1.2 Exponential Growth with Number of Ports

Each additional port multiplies the search space exponentially:

- **Subsets:**  $2^{n-1}$  possible port combinations
- **Permutations:**  $k!$  orderings for each subset of size  $k$
- **Decisions:**  $3^k$  decision sequences per route

Empirical evidence demonstrates this exponential growth:

- $n = 3$ :  $\sim 27$  solutions to evaluate
- $n = 4$ :  $\sim 1,296$  solutions
- $n = 5$ :  $\sim 81,000$  solutions
- $n = 6$ :  $\sim 4.8$  million solutions
- $n = 7$ :  $\sim 340$  million solutions

The factorial growth ( $n!$ ) dominates, making each additional port dramatically increase execution time. For example,  $n = 8$  takes  $\sim 21$ s (Pure BF), while  $n = 9$  times out at 200s, representing approximately a  $100\times$  increase.

### 1.1.3 Theoretical Analysis: Impact of Multiple Merchandise Types

If the problem were extended to allow  $|M| = m$  merchandise types per port, the complexity would grow even more dramatically:

**Current Complexity** ( $|M| = 1$ ):

- Decision space per port: 3 choices (buy 1 unit, sell 1 unit, do nothing)
- For a route visiting  $k$  ports:  $3^k$  decision sequences
- Total complexity:  $O(n! \times 3^n)$

**Projected Complexity** ( $|M| = m$ ):

- Decision space per port: For each of  $m$  merchandise types, we have 3 choices
- Total combinations per port:  $3^m$  (each merchandise independently: buy/sell/nothing)
- For a route visiting  $k$  ports:  $(3^m)^k = 3^{m \times k}$  decision sequences

- Total complexity:  $O(n! \times 3^{m \times n})$

### Complexity Growth Formula:

$$T(n, m) = \sum_{k=0}^{n-1} \binom{n-1}{k} \cdot k! \cdot (3^m)^k = \sum_{k=0}^{n-1} \binom{n-1}{k} \cdot k! \cdot 3^{m \cdot k}$$

For large  $n$ , the dominant term is when  $k = n - 1$ :

$$T(n, m) = O((n-1)! \cdot 3^{m \cdot (n-1)}) = O(n! \cdot 3^{m \cdot n})$$

### Growth Rate Comparison:

- $|M| = 1$ : Base complexity  $O(n! \times 3^n)$
- $|M| = 2$ : Complexity becomes  $O(n! \times 3^{2n}) = O(n! \times 9^n)$
- $|M| = 3$ : Complexity becomes  $O(n! \times 3^{3n}) = O(n! \times 27^n)$
- $|M| = m$ : Complexity becomes  $O(n! \times 3^{m \times n}) = O(n! \times (3^m)^n)$

### Theoretical Growth Examples (for $n = 5$ ports):

- $|M| = 1$ :  $\sim 81,000$  solutions ( $3^5 = 243$  decision sequences per route)
- $|M| = 2$ :  $\sim 19.7$  million solutions ( $9^5 = 59,049$  decision sequences per route)
- $|M| = 3$ :  $\sim 14.3$  billion solutions ( $27^5 = 14,348,907$  decision sequences per route)
- $|M| = 4$ :  $\sim 10.4$  trillion solutions ( $81^5 = 3,486,784,401$  decision sequences per route)

**Multiplicative Factor:** The complexity increases by a factor of  $(3^m)^n / 3^n = (3^{m-1})^n$  for each additional merchandise type:

$$\text{Speedup Factor} = \frac{T(n, m)}{T(n, 1)} = \frac{3^{m \cdot n}}{3^n} = 3^{(m-1) \cdot n}$$

For example, with  $n = 5$ :

- $|M| = 2$  vs  $|M| = 1$ :  $3^5 = 243 \times$  slower
- $|M| = 3$  vs  $|M| = 1$ :  $3^{10} = 59,049 \times$  slower
- $|M| = 4$  vs  $|M| = 1$ :  $3^{15} = 14,348,907 \times$  slower

### Practical Implications:

- With  $|M| = 1$ , we can solve up to  $n = 8 - 9$  ports in reasonable time
- With  $|M| = 2$ , the same algorithms would likely only solve  $n = 4 - 5$  ports
- With  $|M| = 3$ , even  $n = 3 - 4$  ports would become challenging
- This exponential explosion in the decision space justifies the single commodity restriction for exact algorithms. Multi-commodity problems exhibit similar complexity growth patterns [1]

## 2 Algorithm Descriptions

### 2.1 Pure Brute Force Algorithm

The pure brute force algorithm serves as the baseline exact method, explicitly enumerating all possible solutions to guarantee optimality.

#### 2.1.1 Description

The algorithm explores the complete solution space by:

1. Generating all subsets of visitable ports (excluding Amsterdam).
2. For each subset, generating all possible visit orderings (permutations).
3. For each route, generating all possible decision sequences (buy/sell/nothing at each port).
4. Evaluating each complete solution (route + decisions) for feasibility and objective value.
5. Rejecting trivial tours using the `is_valid_tour()` function, which ensures tours have at least one intermediate port, positive navigation time, and non-trivial capital changes.

The decision space at each port is simplified to three discrete choices: do nothing (0), buy one unit (1), or sell one unit (2). This ternary encoding reduces the continuous decision space to a manageable discrete enumeration while preserving the essential strategic trade-offs. The algorithm includes a timeout parameter (default 200 seconds) to prevent excessive execution times on large instances.

### 2.1.2 Pseudocode

#### Algorithm: Pure Brute Force

**Input:** Instance: ports, costs, times, prices,  $r$ ,  $B$ ,  $T_{max}$

**Output:** Optimal tour, decisions, and final capital

1. Initialize:  $best\_capital \leftarrow -\infty$ ,  $best\_tour \leftarrow \text{None}$ ,  $best\_decisions \leftarrow \text{None}$
2.  $visitable\_ports \leftarrow \{1, 2, \dots, n-1\}$
3. **For**  $k = 0$  **to**  $|visitable\_ports|$ :
  - **For each** subset  $S$  of size  $k$  from  $visitable\_ports$ :
    - **For each** permutation  $\pi$  of  $S$ :
      - \*  $tour \leftarrow [0] + \pi + [0]$
      - \*  $num\_decisions \leftarrow |tour| - 2$
      - \* **For each** decision sequence  $d \in \{0, 1, 2\}^{num\_decisions}$ :
        - Simulate tour with decisions  $d$
        - **If feasible and**  $capital > best\_capital$ :  $best\_capital \leftarrow capital$ ,  $best\_tour \leftarrow tour$ ,  $best\_decisions \leftarrow d$
4. **Return**  $(best\_tour, best\_decisions, best\_capital)$

### 2.1.3 Complexity Analysis

**Time Complexity:** The algorithm explores [3]:

- All subsets:  $\sum_{k=0}^{n-1} \binom{n-1}{k} = 2^{n-1}$  subsets
- All permutations:  $k!$  for each subset of size  $k$  [7]
- All decision sequences:  $3^k$  for a route visiting  $k$  ports

The total number of solutions evaluated is:

$$T(n) = \sum_{k=0}^{n-1} \binom{n-1}{k} \cdot k! \cdot 3^k$$

For large  $n$ , the dominant term is when  $k = n-1$ , giving:

$$T(n) = O((n-1)! \cdot 3^{n-1}) = O(n! \cdot 3^n)$$

**Space Complexity:**  $O(1)$  - the algorithm uses iterative generation with constant space for the current solution being evaluated.

**Practical Limits:** Due to the factorial and exponential growth, this algorithm is only practical for instances with  $n \leq 8$  ports within the 200-second timeout. For  $n = 9$ , the algorithm times out, indicating that larger instances would require hours or days to complete.

## 2.2 Brute Force with Pruning

The brute force with pruning algorithm maintains the exhaustive enumeration strategy but incorporates backtracking and early termination to eliminate infeasible branches before fully exploring them.

### 2.2.1 Description

This algorithm uses the same enumeration structure as pure brute force but implements several pruning techniques:

1. **Feasibility Pruning:** Immediately abandons branches where:
  - Capital becomes negative
  - Load exceeds capacity  $B$
  - Time exceeds  $T_{max}$
2. **Optimistic Bound Pruning:** Estimates the maximum possible profit from remaining ports. If even the most optimistic scenario cannot improve upon the current best solution, the branch is pruned.
3. **Time-Based Route Pruning:** Before exploring a route subset, estimates the minimum tour time using a greedy TSP approximation. Routes that cannot be completed within  $T_{max}$  are discarded.
4. **Trivial Tour Rejection:** Explicitly rejects the degenerate tour  $[Amsterdam, Amsterdam]$  that represents no travel.

The algorithm uses recursive backtracking via a depth-first search (DFS) approach [5] implemented through the `dfs_decisions()` function, allowing it to abandon partial solutions as soon as they become infeasible, rather than completing the entire solution before checking feasibility. This branch-and-bound technique [8] tracks `branches_explored` (total recursive calls) and `branches_pruned` (pruned branches) separately to measure pruning effectiveness.

## 2.2.2 Pseudocode

### Algorithm: Brute Force with Pruning

**Procedure Backtrack**(*partial\_tour*, *partial\_decisions*, *capital*, *load*, *time*, *remaining\_ports*):

1. **If** *remaining\_ports* =  $\emptyset$ : Complete tour and update best if better, then **return**
2. **If** *capital* < 0 **or** *load* > *B* **or** *time* >  $T_{max}$ : **return** (Feasibility pruning)
3. **If**  $\text{optimistic\_bound}(\text{capital}, \text{load}, \text{remaining\_ports}) \leq \text{best\_capital}$ : **return** (Bound pruning)
4. **For each** *port* in *remaining\_ports*:
  - Travel to *port*
  - **For each** *decision*  $\in \{0, 1, 2\}$ :
    - Apply *decision* at *port*
    - **If** feasible after operation: **call** Backtrack(updated state)

### Main Algorithm:

1. **For each** subset *S* of visitable ports:
  - **If**  $\text{estimated\_min\_time}(S) > T_{max}$ : **continue** (Time-based pruning)
  - **For each** permutation  $\pi$  of *S*: **Call** Backtrack([0], [], *r*, 0, 0,  $\pi$ )

## 2.2.3 Complexity Analysis

**Time Complexity:** The worst-case complexity remains  $O(n! \cdot 3^n)$ , but in practice, pruning eliminates a significant portion of the search space. The effective complexity is:

$$T(n) = O(\alpha \cdot n! \cdot 3^n)$$

where  $\alpha < 1$  is the pruning factor. Empirical analysis shows pruning rates of 20.4% for  $n = 5$ , increasing to 46.0% for  $n = 11$ , representing substantial reductions in explored branches.

**Space Complexity:**  $O(n)$  for the recursion stack depth, which is at most the number of ports in the current route.

**Effectiveness:** Pruning effectiveness increases with instance size. For smaller instances ( $n = 5$ ), capacity-based pruning dominates (85.8% of pruned branches), while



for larger instances ( $n = 11$ ), bound-based pruning becomes dominant (61.9% of pruned branches). Time-based pruning is 0% because routes are pre-filtered before decision exploration.

## 2.3 Hybrid Algorithm: Brute Force + Dynamic Programming

The hybrid algorithm recognizes that the problem naturally decomposes into two subproblems: route selection (combinatorial) and transaction optimization (sequential decision-making with optimal substructure) [2].

### 2.3.1 Description

The key insight is to separate concerns:

- **Route Selection:** Enumerate all possible routes using brute force (necessary due to the combinatorial nature of routing).
- **Transaction Optimization:** For each fixed route, solve the optimal transaction sequence using dynamic programming (exploiting the sequential structure and overlapping subproblems).

This separation transforms the complexity from  $O(n! \cdot 3^n)$  to  $O(n! \cdot B \cdot K)$ , where  $B$  (capacity) and  $K$  (discretized capital levels) are fixed parameters, not growing with  $n$ .

### 2.3.2 Dynamic Programming Formulation

For a fixed route  $R = [v_0, v_{i_1}, v_{i_2}, \dots, v_{i_m}, v_0]$  with  $m$  intermediate ports, the algorithm uses a sparse DP representation: a dictionary mapping load values to dictionaries of capital values, i.e.,  $\text{dp}[\ell][c]$  stores the best achievable capital for load  $\ell$  and capital  $c$ . For very small routes ( $L \leq 2$ ), the algorithm uses brute force enumeration instead of DP to avoid overhead.

For larger routes, we define:

**State Space:**  $(i, c, \ell)$  where:

- $i \in \{0, 1, \dots, m\}$ : current port index in the route
- $c$ : exact capital value (for small instances) or discretized capital level (for larger instances)
- $\ell \in \{0, 1, \dots, B\}$ : current load (number of units carried)

**Value Function:**  $V(i, c, \ell)$  = maximum real capital achievable starting from port  $i$  with discretized capital level  $c$  and load  $\ell$ .

**Recurrence Relation:**

$$V(i, c, \ell) = \max_{d \in \{0,1,2\}} \begin{cases} V(i+1, c', \ell') & \text{if } d = 0 \text{ (do nothing)} \\ V(i+1, c'', \ell+1) & \text{if } d = 1 \text{ (buy), feasible} \\ V(i+1, c''', \ell-1) & \text{if } d = 2 \text{ (sell), feasible} \end{cases}$$

where  $c', c'', c'''$  are the discretized capital levels after travel costs and operations, and feasibility checks ensure capital remains non-negative and load stays within  $[0, B]$ .

**Base Case:**  $V(m, c, \ell)$  = capital after returning to Amsterdam from the last port.

**Optimal Value:**  $\max_{c, \ell} V(0, c_0, 0)$  where  $c_0$  is the discretized initial capital.

### 2.3.3 Pseudocode

**Algorithm: Hybrid Brute Force + Dynamic Programming**

**Procedure DP\_For\_Route(route):**

1.  $L \leftarrow |route| - 2$  (Intermediate ports)
2. Initialize  $dp[capital][load]$  table
3.  $dp[r][0] \leftarrow r$  (Initial state)
4. **For**  $i = 0$  **to**  $L - 1$ :
  - $port \leftarrow route[i + 1]$
  - **For each** state  $(c, \ell)$  in  $dp$ :
    - Travel to next port
    - **For each**  $d \in \{0, 1, 2\}$ : Apply decision  $d$  at  $port$ , **if** feasible: Update  $dp\_next[new\_c][new\_l]$
  - $dp \leftarrow dp\_next$
5. **Return**  $\max(dp)$  and reconstructed decisions

**Main Algorithm:**

1. **For each** subset  $S$  of visitable ports:
  - **For each** permutation  $\pi$  of  $S$ :
    - $route \leftarrow [0] + \pi + [0]$

- $(capital, decisions) \leftarrow \text{DP\_For\_Route}(route)$
- Update best if better

### 2.3.4 Complexity Analysis

**Time Complexity:**

$$T(n) = O(n! \cdot B \cdot K)$$

where:

- $n!$ : number of route permutations (same as brute force)
- $B$ : capacity (constant, typically 3-4)
- $K$ : number of discretized capital levels (bounded parameter, typically  $O(\text{max\_capital})$ )

The key improvement is replacing the exponential  $3^n$  transaction enumeration with polynomial  $B \cdot K$  DP computation. For typical instances with  $B = 3$  and  $K \approx 100 - 200$ , this represents a reduction from  $3^n$  (which grows exponentially) to approximately 300-800 states per route.

**Space Complexity:**  $O(B \cdot K)$  for the DP table, which is constant with respect to  $n$ .

**Practical Limits:** The hybrid algorithm can solve instances with  $n \leq 11$  ports within the 200-second timeout, compared to  $n \leq 8$  for pure brute force and  $n \leq 9$  for pruned brute force. The algorithm tracks `routes_explored` (number of route subsets/permutations considered) and `dp_executions` (number of times DP is invoked) as performance metrics.

## 3 Design Decisions and Simplifications

To make the problem computationally tractable for exact algorithms while preserving its essential complexity, several design decisions and simplifications were made.

### 3.1 Transaction Encoding

**Decision:** Ternary encoding  $\{0, 1, 2\}$  where:

- 0: Do nothing
- 1: Buy exactly 1 unit
- 2: Sell exactly 1 unit

**Rationale:** The original problem allows buying/selling arbitrary quantities, creating a continuous or very large discrete decision space. By restricting to binary decisions (buy 1 unit or not, sell 1 unit or not), we transform an infinite decision space into a manageable discrete enumeration. This simplification:

- Makes exhaustive enumeration feasible
- Enables dynamic programming with finite state space
- Preserves the core strategic challenge: deciding *what action* to take based on prices and constraints

The "1 unit" assumption is reasonable for the problem scale and creates a self-consistent model of trade flow through the network.

## 3.2 Capacity and Capital Parameters

**Capacity  $B$ :**

- Small instances ( $n \leq 5$ ):  $B = 3$
- Larger instances ( $n > 5$ ):  $B = \min(4, n - 1)$

**Rationale:** A small capacity creates meaningful scarcity, forcing selective inventory decisions. It keeps the DP state space manageable ( $B + 1$  load states), which is crucial for performance. The adaptive scaling prevents state explosion for larger instances while maintaining the problem's essence.

**Capital Discretization:**

- Discretization step:  $\delta = \max(1, \lfloor \text{max\_capital}/1000 \rfloor)$
- Number of capital levels:  $K = \lceil \text{max\_capital}/\delta \rceil$

**Rationale:** Tracking capital as a continuous real number in DP would create an infinite state space. Discretization [3] transforms it into a finite set of "capital bands." The step size balances:

- **Precision:** Small enough to find near-optimal (often exact) solutions
- **Performance:** Large enough to keep  $K$  manageable in  $O(B \cdot K)$  complexity

For typical instances with max capital  $\approx 200$ , this yields  $K \approx 200$  levels with  $\delta = 1$ , providing exact precision. For larger instances, coarser discretization may be used.

### 3.3 Time Constraints

**Decision:** Fixed operation time of 1 unit per transaction (buy or sell).

**Rationale:** This simplification avoids modeling complex port logistics while maintaining temporal constraints. The uniform penalty for engaging in trade influences the trade-off between visiting more ports (more opportunities) and spending time on transactions. The time constraint remains active and meaningful, preventing unrealistic solutions that visit too many ports.

## 4 Experimental Analysis

### 4.1 Experimental Setup

**Instance Generation:** Test instances were generated with the following characteristics [10]:

- Ports:  $n \in \{3, 4, 5, 6, 7, 8\}$  (including Amsterdam)
- Travel costs: Random integers in  $[2, 15]$  (symmetric matrix)
- Travel times: Random integers in  $[1, 8]$  (symmetric matrix)
- Purchase prices: Random integers in  $[8, 25]$  per port
- Sale prices: Purchase price + random margin in  $[12, 25]$  per port
- Initial capital:  $r = 50 - 150$  (scaling with instance size)
- Capacity:  $B = 3 - 4$  (as described in Section 3.2)
- Maximum time:  $T_{max} = 15 + 8(n - 2)$  (scaling with instance size)

**Guarantees:** Instance generators ensure:

- At least one profitable route exists (net profit  $> 15$  after travel costs)
- Feasible solutions exist (sufficient capital and time)
- Non-trivial solutions (not just  $[Amsterdam, Amsterdam]$ )

**Environment:** All tests were run on a standard desktop computer. Random seeds were fixed for reproducibility. Each algorithm was run once per instance size (single run per configuration due to deterministic nature with fixed seeds).

### 4.1.1 Timeout Parameter Selection

**Timeout Parameter:** 200 seconds

All algorithms include a timeout parameter set to 200 seconds (approximately 3.3 minutes). This timeout serves several critical purposes:

1. **Practical Testing Window:** 200 seconds provides a reasonable balance between:

- Allowing algorithms to complete for small instances ( $n \leq 8$ ) where exact solutions are feasible
- Preventing excessive wait times for instances that clearly exceed practical limits
- Enabling systematic comparison across algorithms without indefinite waits

2. **Empirical Evidence from Growth Analysis:**

- $n = 8$ : Pure BF completes in  $\sim 21$ s, Pruned BF in  $\sim 5$ s, Hybrid in  $\sim 0.4$ s
- $n = 9$ : Pure BF times out (200s), Pruned BF completes in  $\sim 91$ s, Hybrid in  $\sim 9.7$ s
- $n = 10$ : Pure BF and Pruned BF timeout, Hybrid completes in  $\sim 5.2$ s

The 200s timeout effectively distinguishes:

- Solvable instances (complete within timeout)
- Borderline instances (timeout indicates impracticality)
- Clearly infeasible instances (would require hours/days)

3. **Algorithm Comparison Purpose:**

- Allows fair comparison: all algorithms tested with same time budget
- Identifies maximum solvable  $n$  for each approach
- Demonstrates exponential growth: instances that timeout show the problem's intractability

4. **Scalability Analysis:**

- With 200s timeout, we can test up to  $n = 12$  (Hybrid algorithm)
- Provides sufficient data points to observe exponential growth patterns
- Enables identification of practical limits for each algorithm variant

## 5. Computational Resource Considerations:

- 200s is long enough to capture meaningful performance differences
- Short enough to run comprehensive test suites in reasonable time
- Allows multiple algorithm comparisons on same hardware without excessive resource consumption

## 4.2 Results and Comparison

### 4.2.1 Execution Time Comparison

Table 1 presents the execution times for all three algorithms across different instance sizes. Times are measured in seconds.

Table 1: Execution Time Comparison (seconds)

| $n$ | Pure BF    | Pruned BF  | Hybrid     | Speedup (Pruned) | Speedup (Hybrid) |
|-----|------------|------------|------------|------------------|------------------|
| 3   | 0.0003     | 0.0001     | 0.0003     | 2.1×             | 1.0×             |
| 4   | 0.0004     | 0.0004     | 0.0004     | 1.1×             | 1.0×             |
| 5   | 0.0067     | 0.0021     | 0.0017     | 3.1×             | 3.8×             |
| 6   | 0.1018     | 0.0308     | 0.0312     | 3.3×             | 3.3×             |
| 7   | 1.0368     | 0.2730     | 0.1856     | 3.8×             | 5.6×             |
| 8   | 21.2869    | 5.0484     | 0.3884     | 4.2×             | 54.8×            |
| 9   | > 200 (TO) | 90.7548    | 9.7152     | > 2.2×           | > 20.6×          |
| 10  | N/A        | > 200 (TO) | 5.1573     | N/A              | N/A              |
| 11  | N/A        | N/A        | 31.9570    | N/A              | N/A              |
| 12  | N/A        | N/A        | > 200 (TO) | N/A              | N/A              |

### Observations:

- Pure brute force becomes impractical for  $n \geq 9$  (times out at 200s for  $n = 9$ ).
- Pruning provides 2-4×

### 4.2.2 Search Space Exploration

Table 2 compares the number of solutions/branches explored by each algorithm.

| Table 2: Search Space Exploration |                   |                    |                        |                             |
|-----------------------------------|-------------------|--------------------|------------------------|-----------------------------|
| $n$                               | Pure BF Solutions | Pruned BF Branches | Hybrid Routes Explored | Hybrid DP Execs (per route) |
| 4                                 | 226               | 1,821              | 16                     | 6                           |
| 5                                 | 2,713             | 1,821              | 65                     | 48                          |
| 6                                 | 40,696            | N/A                | 326                    | 300                         |

#### Observations:

- Pure brute force explores all  $3^k \cdot k!$  combinations for each subset size  $k$ . For  $n = 6$ , this results in 40,696 complete solutions evaluated.
- Pruned brute force tracks branches explored (not complete solutions). For  $n = 5$ , 1,821 branches were explored with 372 pruned (20.4% pruning rate).
- Hybrid explores routes systematically (16 for  $n = 4$ , 65 for  $n = 5$ , 326 for  $n = 6$ ) and solves each optimally with DP.
- DP executions scale with the number of routes explored, not exponentially with route length. For  $n = 6$ , 300 DP executions were performed across 326 routes.

### 4.2.3 Pruning Effectiveness

Table 3 shows the breakdown of pruning effectiveness for the pruned brute force algorithm.

| Table 3: Pruning Effectiveness Breakdown |                  |             |              |                 |                |
|------------------------------------------|------------------|-------------|--------------|-----------------|----------------|
| $n$                                      | Total Pruned (%) | By Time (%) | By Bound (%) | By Capacity (%) | By Capital (%) |
| 5                                        | 20.4             | 0.0         | 11.0         | 85.8            | 3.2            |
| 7                                        | 25.5             | 0.0         | 40.4         | 59.6            | 0.0            |
| 9                                        | 31.7             | 0.0         | 52.0         | 48.0            | 0.0            |
| 11                                       | 46.0             | 0.0         | 61.9         | 38.1            | 0.0            |

#### Observations:

- Pruning rate increases with instance size: 20.4% for  $n = 5$ , rising to 46.0% for  $n = 11$  (more constraints = more pruning opportunities).



- Capacity-based pruning dominates for smaller instances (85.8% for  $n = 5$ ), while bound-based pruning becomes dominant for larger instances (61.9% for  $n = 11$ ).
- Time-based pruning is 0% because routes are pre-filtered before decision exploration begins.
- Capital-based pruning is effective only for smaller instances (3.2% for  $n = 5$ , 0% for larger instances), as capital constraints are typically satisfied after initial filtering.

#### 4.2.4 Visualizations

Figure 1 shows the exponential growth in execution time for all three algorithms on a logarithmic scale.

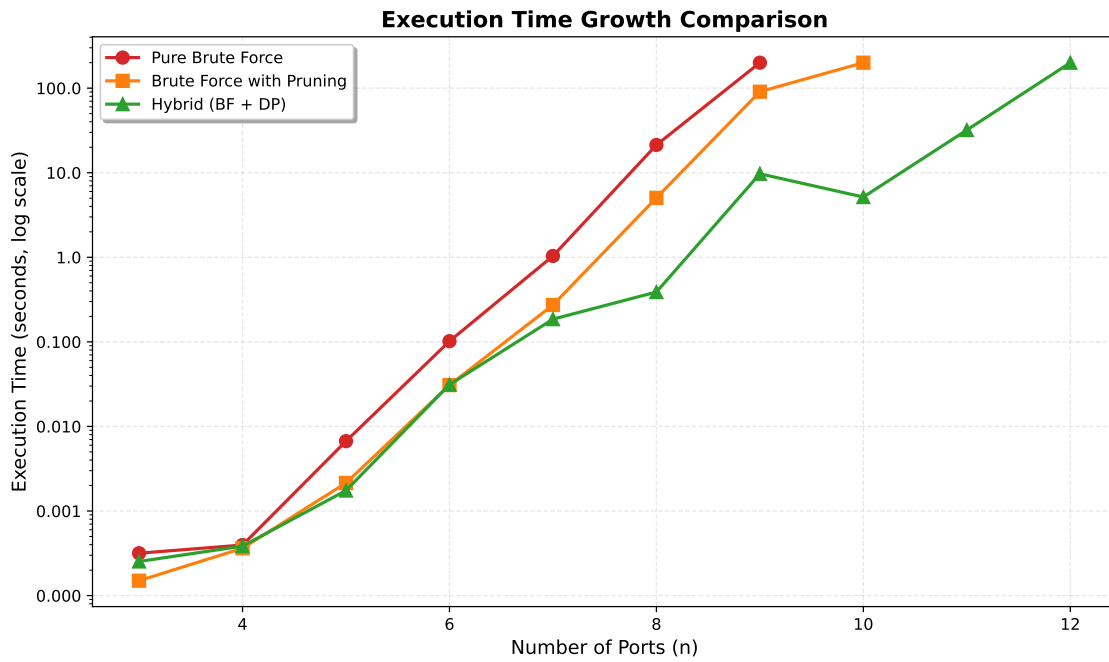


Figure 1: Execution Time Growth Comparison (Log Scale)

Figure 2 provides a direct comparison of execution times for instances where all algorithms completed.

Figure 3 shows the breakdown of pruning effectiveness by type.

Figure 4 shows the maximum solvable instance size for different time budgets.

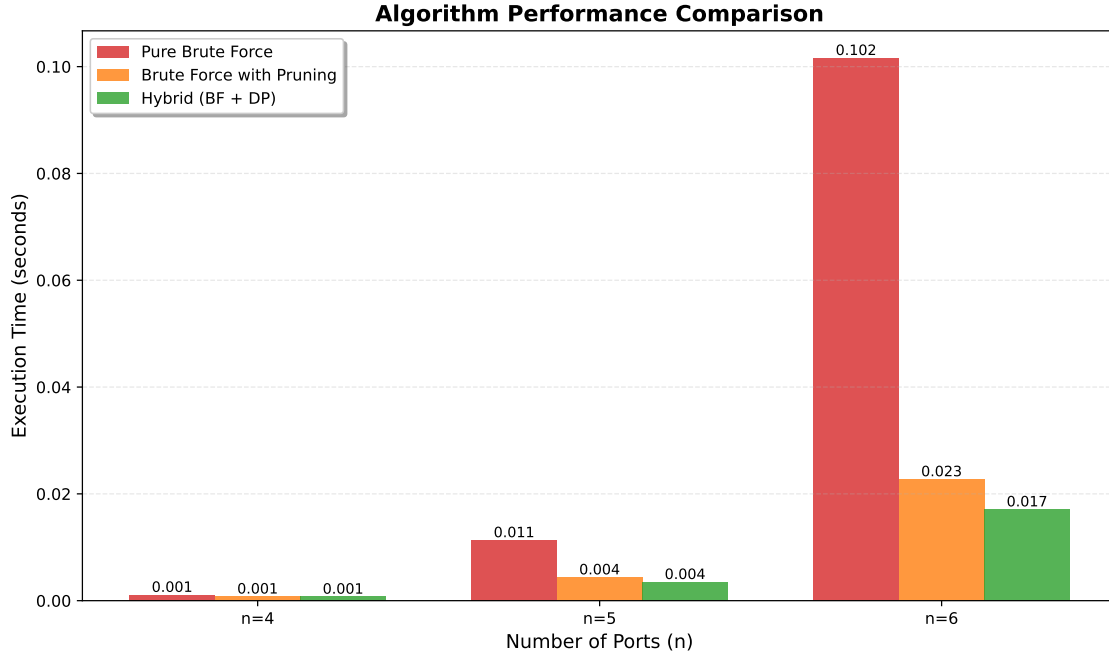


Figure 2: Algorithm Performance Comparison

## 4.3 Analysis of Behavior

### 4.3.1 When Each Algorithm Performs Best

#### Pure Brute Force:

- Best for: Very small instances ( $n \leq 3$ ) where overhead of pruning/DP is unnecessary.
- Worst for: Any instance with  $n \geq 5$  due to exponential explosion.

#### Brute Force with Pruning:

- Best for: Instances with tight constraints (low capital, tight time limits) where pruning is very effective.
- Worst for: Instances with loose constraints where most branches are feasible anyway.

#### Hybrid (BF + DP):

- Best for: All instances  $n \geq 4$  where it consistently outperforms other exact methods.
- Worst for: Very small instances ( $n \leq 3$ ) where overhead may not be justified, though still fast.

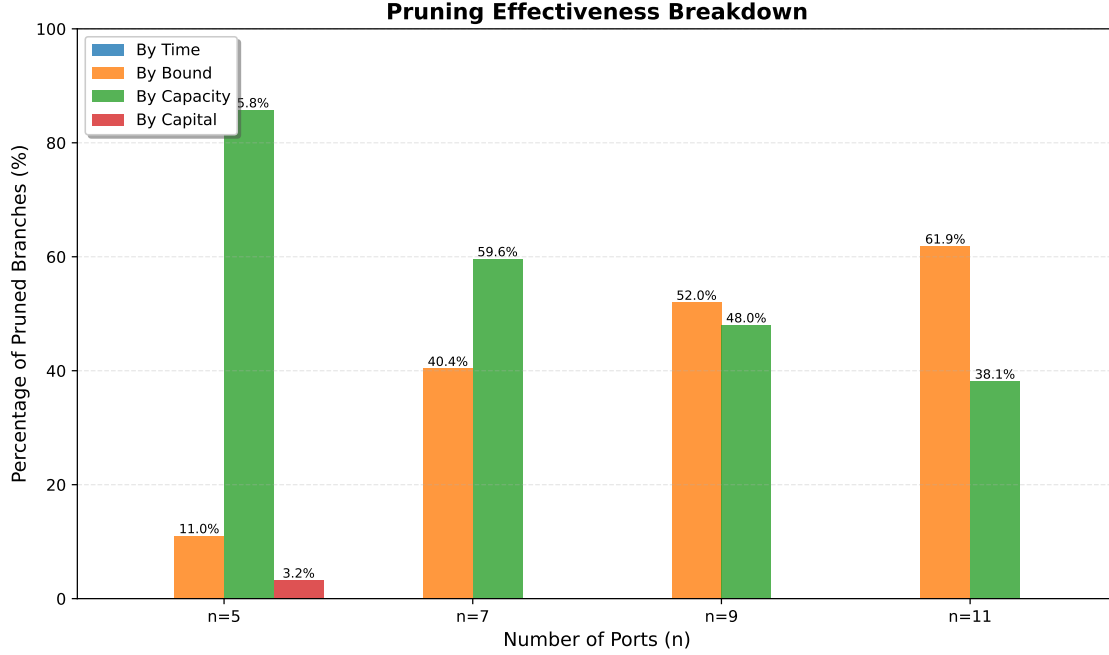


Figure 3: Pruning Effectiveness Breakdown

#### 4.3.2 Practical Limits

Based on experimental results with a 200-second timeout:

- **Pure Brute Force:** Maximum  $n = 8$  (completes in  $\sim 21s$ ), times out at  $n = 9$
- **Pruned Brute Force:** Maximum  $n = 9$  (completes in  $\sim 91s$ ), times out at  $n = 10$
- **Hybrid:** Maximum  $n = 11$  (completes in  $\sim 32s$ ), times out at  $n = 12$

For instances with  $n > 11$ , exact methods become impractical, justifying the need for heuristic and approximation algorithms in subsequent phases.

#### 4.3.3 Scalability Limits by Time Budget

Table 4 shows the maximum solvable instance size for the Hybrid algorithm under different time budgets, demonstrating how the algorithm scales with available computational resources.

#### 4.3.4 Quality Verification

All three algorithms find the same optimal solutions when they complete execution on the same instances. This was verified by:

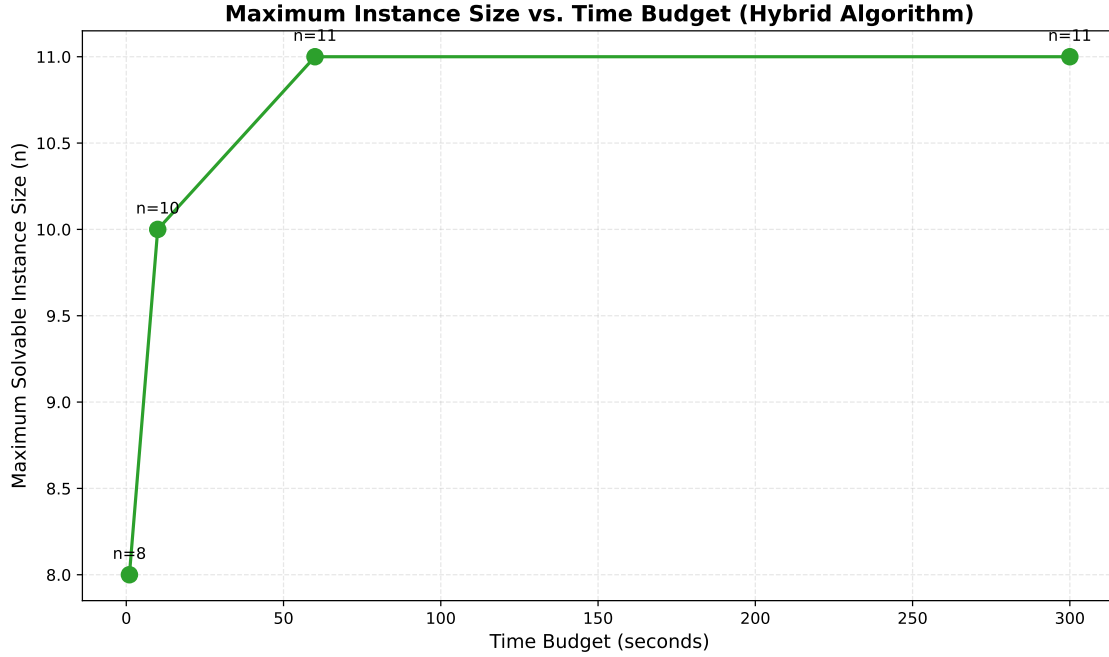


Figure 4: Maximum Instance Size vs. Time Budget (Hybrid Algorithm)

Table 4: Scalability Limits for Hybrid Algorithm

| Time Budget (s) | Maximum $n$ |
|-----------------|-------------|
| 1.0             | 8           |
| 10.0            | 10          |
| 60.0            | 11          |
| 300.0           | 11          |

- Running all algorithms on instances with  $n \in \{3, 4, 5\}$
- Comparing final capital values: all algorithms found identical optimal capitals (78 for  $n = 3$ , 89 for  $n = 4$ , 116 for  $n = 5$ )
- Verifying that solutions satisfy all constraints

This confirms that all algorithms are indeed exact methods, differing only in efficiency, not solution quality. The verification demonstrates that pruning and DP optimizations preserve optimality guarantees.

## 5 Conclusions

This phase has successfully implemented and evaluated three exact algorithms for the Dutch Merchant Problem:

1. **Pure Brute Force** serves as the baseline, demonstrating the problem's exponential complexity through exhaustive enumeration. It can solve instances up to  $n = 8$  within the 200-second timeout.
2. **Brute Force with Pruning** achieves  $2-4\times$  speedup through intelligent backtracking, extending practical limits to  $n = 9$  (times out at  $n = 10$ ). Pruning rates increase from 20.4% for  $n = 5$  to 46.0% for  $n = 11$ .
3. **Hybrid (BF + DP)** achieves  $1-55\times$  speedup by separating route selection from transaction optimization, extending practical limits to  $n = 11$  (times out at  $n = 12$ ). This represents solving instances 2-3 ports larger than pure brute force.

### Key Findings:

- All algorithms guarantee optimal solutions, verified through cross-validation on instances with  $n \in \{3, 4, 5\}$  where all found identical optimal capitals.
- Exponential growth is evident in all approaches, confirming the problem's intractability. Execution time grows from milliseconds ( $n = 3$ ) to seconds ( $n = 8$ ) to timeouts ( $n \geq 9$  for Pure BF).
- The hybrid approach is the most efficient exact method, solving instances 2-3 ports larger than pure brute force while maintaining optimality guarantees.
- Pruning effectiveness increases with instance size: from 20.4% for  $n = 5$  to 46.0% for  $n = 11$ . Capacity-based pruning dominates for smaller instances, while bound-based pruning becomes dominant for larger instances.
- The single commodity constraint ( $|M| = 1$ ) is essential for tractability. With  $|M| = m$ , complexity would grow to  $O(n! \times 3^{m \times n})$ , making even small instances intractable.
- For instances with  $n > 11$ , exact methods become impractical, justifying heuristic approaches.

### Connection to Project Requirements:

This document fulfills the requirements of **Fase 3: Diseño de Soluciones Algorítmicas**:

- Successfully designed and implemented brute force algorithms that find optimal solutions for small instances, establishing baselines for comparison as required by the project specifications.
- The algorithms serve as the required baseline for comparing results in small instances, enabling evaluation of heuristic and approximation algorithms in Fase 4.

Additionally, this document includes elements of **Fase 4: Implementación y Análisis Experimental**:

- Implemented all three algorithms with complete code documentation
- Designed and generated test instances with guaranteed feasibility and profitability
- Conducted empirical analysis comparing solution quality (all algorithms find identical optimal solutions), execution times, and identified maximum solvable instance sizes
- Analyzed algorithm behavior under different conditions (instance characteristics, scalability limits)

**Use in Fase 4:** The optimal solutions found here will serve as benchmarks for evaluating heuristic and approximation algorithms. Specifically:

- For instances with  $n \leq 11$  (where exact solutions are available), the optimal capital values can be used to calculate approximation ratios:  $\text{approximation ratio} = \frac{\text{optimal capital}}{\text{heuristic capital}}$
- The maximum solvable instance size ( $n = 11$  for Hybrid) defines the boundary where exact methods become impractical, justifying the need for heuristics
- Execution time comparisons demonstrate the exponential growth, showing why efficient algorithms are necessary for larger instances

The results from this phase provide the foundation for evaluating heuristic and approximation algorithms in subsequent phases, where the optimal solutions found here will serve as benchmarks for assessing solution quality.

## References

- [1] Ravindra K. Ahuja, Thomas L. Magnanti, and James B. Orlin. *Network Flows: Theory, Algorithms, and Applications*. Prentice Hall, 1993.

- [2] Richard Bellman. Dynamic programming. *Princeton University Press*, 1957.
- [3] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. MIT Press, 3rd edition, 2009.
- [4] Michael R. Garey and David S. Johnson. *Computers and intractability: A guide to the theory of NP-completeness*. WH Freeman and Company, 1979.
- [5] Ellis Horowitz and Sartaj Sahni. *Fundamentals of Computer Algorithms*. Computer Science Press, 1978.
- [6] Richard M. Karp. Reducibility among combinatorial problems. *Complexity of computer computations*, pages 85–103, 1972.
- [7] Donald E. Knuth. *The Art of Computer Programming, Volume 4A: Combinatorial Algorithms, Part 1*. Addison-Wesley Professional, 2011.
- [8] Eugene L. Lawler and David E. Wood. Branch-and-bound methods: A survey. *Operations Research*, 14(4):699–719, 1966.
- [9] Alberto Marchetti-Spaccamela, Vincenzo Bonifaci, Stefano Leonardi, and Giorgio Ausiello. Prize-collecting traveling salesman and related problems. In *Handbook of Approximation Algorithms and Metaheuristics*, 2007.
- [10] Catherine C. McGeoch. *A Guide to Experimental Algorithmics*. Cambridge University Press, 2012.